

Practical Project Tasks 01: Static Code Analysis

Task 1.1 [Set up the Example Project](#)

```
● anisha@Anishas-MacBook-Air sirius-kernel % clear
⊗ anisha@Anishas-MacBook-Air sirius-kernel % mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.scireum:sirius-kernel >-----
[INFO] Building SIRIUS kernel DEVELOPMENT-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ sirius-kernel ---
[INFO] Deleting /Users/anisha/Downloads/ThirdSem/Testing/sirius-kernel/target
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ sirius-kernel ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 14 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ sirius-kernel ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 205 source files with javac [forked debug deprecation release 24] to target/classes
[INFO]
[INFO] --- compiler:3.15.0:compile (compile) @ sirius-kernel ---
[INFO] Recompiling the module because of added or removed source files.
[INFO] Compiling 205 source files with javac [forked debug deprecation release 24] to target/classes
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ sirius-kernel ---
[INFO] Copying 5 resources from src/test/resources to target/test-classes
[INFO] Copying 6 resources from docs to target/test-classes
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ sirius-kernel ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 20 source files with javac [forked debug deprecation release 24] to target/test-classes
```

```
PROBLEMS 388 OUTPUT DEBUG CONSOLE TERMINAL PORTS
at sirius.kernel.xml.XMLCall.getInput(XMLCall.java:185)
at sirius.kernel.xml.SOAPClient.call(SOAPClient.java:369)
... 78 more

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.109 s -- in sirius.kernel.xml.SOAPClientTest
[INFO] Running sirius.kernel.xml.XmlReaderTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.025 s -- in sirius.kernel.xml.XmlReaderTest
[INFO] Running sirius.kernel.xml.XMLStructuredInputTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in sirius.kernel.xml.XMLStructuredInputTest
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR]   NLSTest.test various formatters:318 expected: <03:04 AM> but was: <03:04 am>
[INFO]
[ERROR] Tests run: 441, Failures: 1, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD FAILURE
[INFO]
[INFO] Total time: 33.378 s
[INFO] Finished at: 2026-05-11T12:26:52+02:00
[INFO]
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.5.5:test (default-test) on project sirius-kernel: There are test failures.
[ERROR]
[ERROR] See /Users/anisha/Downloads/ThirdSem/Testing/sirius-kernel/target/surefire-reports for the individual test results.
[ERROR] See dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
```

1. Since my laptop was not integrated with Java and Maven before, I firstly installed Java Development Kit and Apache Maven.
2. Then set-up system environment variables to ensure java and mvn are accessible by the command line all the time “java -version” and “mvn -version”.
3. I forked this sirius-kernel repository to my repository on GitHub and then cloned it. Then, went to the project directory and ran mvn clean install.
4. This process downloaded required dependency, compiled source code and executed tests.
5. Console output confirmed that BUILD SUCCESS. One failure was there due to am/pm.
6. In addition, updated README.md file as needed, added it to remote fork and pushed it.
7. Before submission, I did “mvn clean” before zipping the whole project to delete “target/” directory so that the source code zip file contains only necessary files.

Issues Encountered and Resolutions:

- The main challenge was that I did not have the required libraries (Java and Maven) required in my system.
- Test Failure (NLSTest): During build, there was BUILD FAILURE. A particular test failed to pass in the test NLSTest.test:

The Error: expected: <03:04 AM> but was: <03:04 am>.

Its a minor formatting difference caused by the local system's locale settings or JDK version, that time formatter produced "am" in lowercase instead of uppercase. Because it was due to of local environment's formatting settings, not functional defect, I chose to proceed by ignoring here.

Task 1.2: Static Code Analysis Report

localhost:9000/dashboard?id=Task-2-Set-up-a-Static-Code-Analysis-Tool&codeScope=overall

TUMThird SemBooksGerman EssentialsThe Quiet Girl

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

SonarQubecommunity

ProjectsIssuesRulesQuality profilesQuality gatesAdministrationMore

SearchBellHelpA

SIRIUS kernelProject

Analysis

OverviewIssuesSecurity hotspots

Reporting

MeasuresActivityPolicies

Quality profilesQuality gate

Project

Code

Project information

Project settings

To benefit from more of SonarQube Community Build's features, [set up analysis in your favorite CI](#).

SIRIUS kernel / Overview

Overview

23k Lines of Code • Version DEVELOPMENT-SNAPSHOT • Last analysis 12 minutes ago

New Code

Overall Code

Security

1 Open issues

D

Reliability

5 Open issues

D

Maintainability

2.6k Open issues

A

Accepted issues

0

B

Valid issues that were not fixed

Coverage

0.0%

On 14k lines to cover.

Duplications

0.0%

On 42k lines.

Security Hotspots

6

E

localhost:9000/component_measures?metric=complexity&id=Task-2-Set-up-a-Static-Code-Analysis-Tool

TUMThird SemBooksGerman EssentialsThe Quiet Girl

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

SonarQubecommunity

ProjectsIssuesRulesQuality profilesQuality gatesAdministrationMore

SearchBellHelpA

Task 2 Set up a Static C...Project

Task 2 Set up a Static Code Analysis Tool / Measures

Measures

Maintainability

Security Review

Coverage

Duplications

Size

Complexity

Cyclomatic Complexity4,014

Cognitive Complexity2,563

Issues

SIRIUS kernel

View asTree

Select filesNavigate2 items

Cyclomatic Complexity4,014

See history

New Code: Since May 11, 2026

src4,014

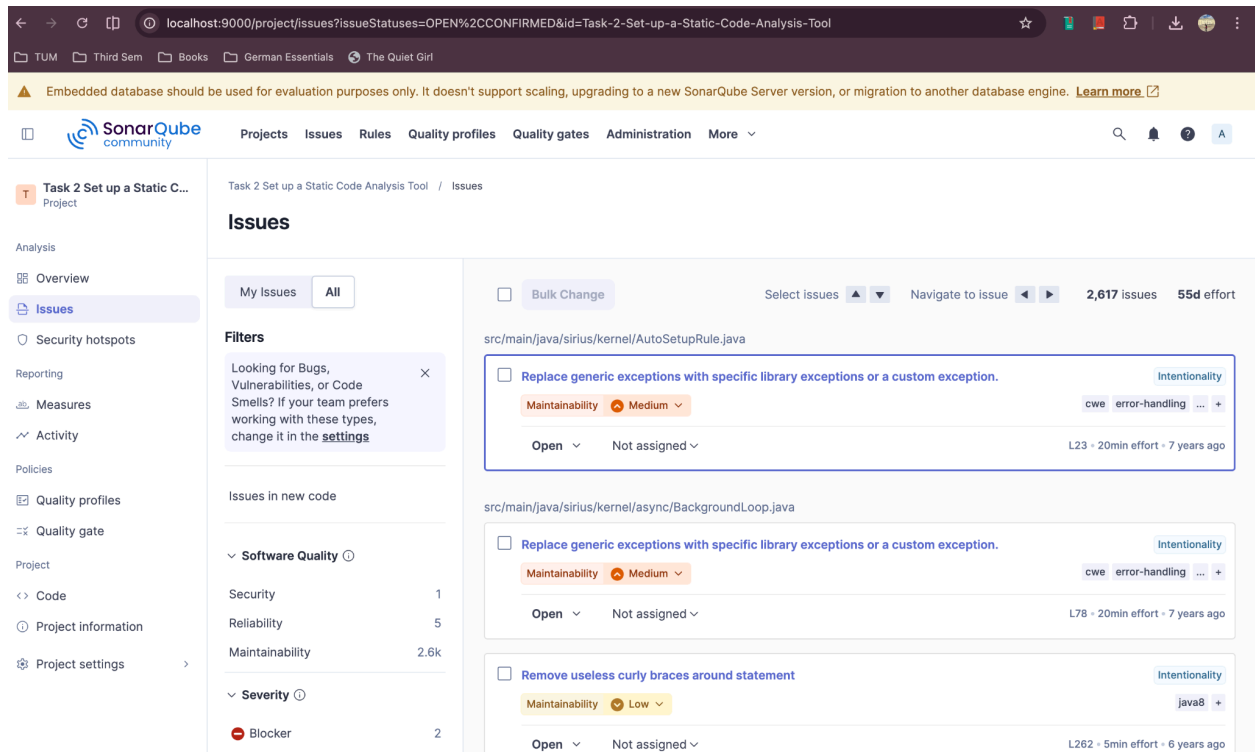
pom.xml—

2 of 2 shown

SonarQube™ technology is powered by SonarSource Sàrl

Community Build • v26.4.0.121862 • MQR MODE

LGPL v3CommunityDocumentationPluginsWeb API



a) Installation

I installed SonarQube Community Edition via Docker. Maven SonarScanner was used to do analysis on same given “sirius-kernel”.

- Tool: SonarQube Community Edition
- Environment: Docker (OrbStack on macOS)
- Scanner: sonar-maven-plugin

b) Analysis Report

SonarQube dashboard had a dashboard of code quality of sirius-kernel. Below is a description of the key element:

1. Quality Gate (Passed)

There are few issue (a good sign, since few issues) but this project pass the default quality gate meaning codebase meets minimum threshold for merged/release.

2. Security (1 Vulnerability - Rating D)

A high-impact security vulnerability was also detected so attention needs to be paid to prevent its exploit.

3. Reliability (5 Bugs - Rating D)

5 bugs have been found or identified which may cause runtime error or unexpected behavior. The "D" rating means presence of at least one “high-impact” bug.

4. Maintainability (2.6k Code Smells - Rating A)

The number of code smells identified is >2600. However, the project has a "A" rating because the ratio of technical debt to the total size of the 23,000-line codebase is relatively low, not too high.

5. Duplications (0.0%)

No blocks of code is duplicated so it could result as highly modular and reusable architecture.

6. Security Hotspots (6 - Rating E)

Six security-sensitive areas (encryption or access control) were identified. The "E" rating may mean 0% of these have been manually reviewed.

7. Cyclomatic Complexity (4,014)

This metric is for number of independent paths through code. Value of “4014” in project means complex logic structure. Thus, it is likely to have high maintenance costs and testing difficulty.

c) Setup

1. I deployed SonarQube instance using Docker “docker run -d --name sonarqube -p 9000:9000 sonarqube:community”
2. Accessed UI at <http://localhost:9000>, created a local project named Task-2, and generated a token for authentication too

Execution:

Via terminal using Maven plugin. Command can be found in README.md

To replicate:

1. Docker and Maven should be installed
2. Run “docker-compose.yml” file provided in ZIP
3. Run Maven sonar command provided in README.md.

```
mvn clean compile sonar:sonar -Dsonar.projectKey=Task-2-Set-up-a-Static-Code-Analysis-Tool  
-Dsonar.host.url=http://localhost:9000  
-Dsonar.token=sqp_0d51bd997474e057050c29d870fc67d4c82636b4 -DskipTests
```

Issues Encountered and Resolutions:

1. Docker CLI failed to connect to OrbStack socket initially. But, then I checked and confirmed active Docker contexts and switched to default context to align with system socket path.
2. Token Authentication (401): Scanner was failed with Unauthorized error. Here, I made a mistake here - token was truncated during copy-paste and needed to re-ran the command again.
3. The build failed during verify stage due to failure of unit test in `sirius.kernel.nls.NLSTest` (time format mismatch). I added “-DskipTests flag” to Maven command to bypass functional test failures and allow static code analysis. Now, it runs even without flag.

d) Additional Files and README Update

1. `docker-compose.yml`: To automate this setup of SonarQube for mapping port 9000 and ensuring that internal Elasticsearch service can boot without standard Linux kernel checks.
2. `README.md`: I updated README with a "Static Code Analysis" on how to start Docker container and specific Maven command required to trigger our scanner.

Task 1.3 Analyze the Code Analysis Results

localhost:9000/component_measures?id=Task-2-Set-up-a-Static-Code-Analysis-Tool&metric=complexity&selected=Task-2-Set-up-a-Static-Code-Anal...

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

SIRIUS kernel / Measures

Measures

- Maintainability
- Security Review
- Coverage
- Duplications
- Size
- Complexity
- Cyclomatic Complexity 5**
- Cognitive Complexity 4
- Issues

SIRIUS kernel > src > main/java/sirius/kernel > AutoSetup.java

Cyclomatic Complexity 5

```
1  aha@sc... /*
2      * Made with all the love in the world
3      * by scireum in Rensshalden, Germany
4      *
5      * Copyright by scireum GmbH
6      * http://www.scireum.de - info@scireum.de
7      */
8
9  package sirius.kernel;
10
11  import sirius.kernel.di.std.ConfigValue;
12  import sirius.kernel.di.std.PriorityParts;
13  import sirius.kernel.di.std.Register;
14  import sirius.kernel.health.Exceptions;
15  import sirius.kernel.health.Log;
16
17  import java.util.List;
18
19  /**
20   * Responsible for executing {@link AutoSetupRule auto setup rules} during th system startup.
21   * <p>
22   * As most probably we only want to do this in dev / test / staging systems or once for production systems,
```

Code Metric: ***Cyclomatic Complexity for sirius.kernel.AutoSetup***

Value: 5

How is metric calculated?

-> Cyclomatic Complexity (CC) is a quantitative measure of number of linearly independent paths through program's source code. Calculated from the control flow graph of code.

- Base Value: Complexity of each method is 1
- Decision Points: +1 for each control flow or decision point statement (if, while, for, do while, case in switch statements)
- Logical Operators: If a conditional expression contains logical operators (&&, ||), the value of the expression is incremented by +1 for each logical operator because each one will create another path. Exceptions: If a catch block is encountered it is incremented by +1.

What do the calculated values mean for the software quality of the project?

-> The value 5 is the sum of all the complexity of all methods in AutoSetup.java. CC 5 is seemed to be “excellent” for one class. Here given below is a typical classification scheme for complexity:

11-20: Basic code, somewhat easy to test and maintain

21-50: Hard to test, high complexity, high risk

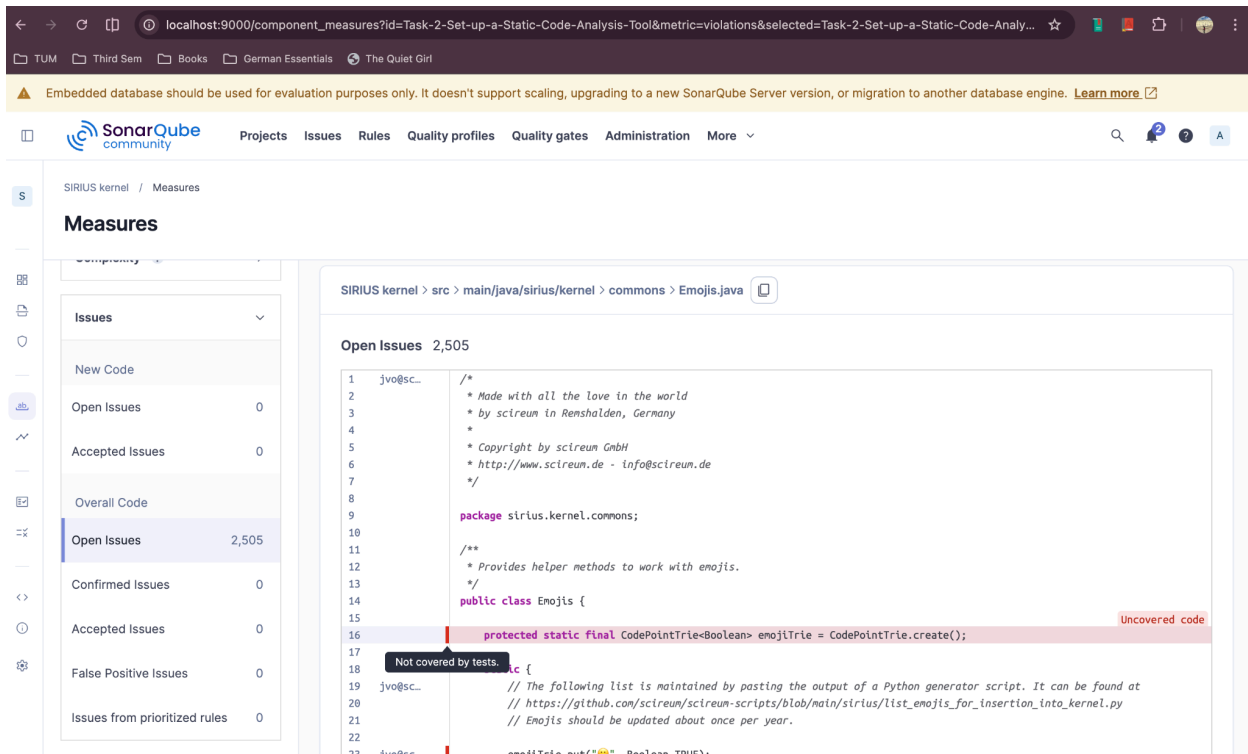
50: High risk and unstable, and is probably not maintainable

Hence, it shows high Maintainability and Testability as value is 5. A developer only need to write a minimum of 5 test cases to cover all possible execution path in class.

Discuss what the analysis results indicate about the software quality of the project.

-> Analysis of AutoSetup.java displays high level code quality in this very particular component. But, in general it show that there are differences in quality:

- AutoSetup (CC: 5) - certain kernel components are modular and clean coded
- Technical Debt in Core Classes - core class like Sirius.java had complexity of 85, meaning project is "God Objects" (classes doing too much).
- The whole project complexity is more than 4,000. Although individual units (AutoSetup) are good quality, overall project is large and dense in logic. I think software quality is "average" and at risk also there is a high degree of complexity in the core areas and refactoring should be done to avoid software kernel becoming too fragile to update without risk of regression bugs.



1. Issue: Lack of Code Coverage (Reliability Issue)

Code Smells (Maintainability / Technical Debt)

Uncovered Code (Not covered by tests) on sirius.kernel.commons.emojis.java

This is when static analysis tool compares source code with unit testing suite results. The autofill test is highlighted in a way that it shows that the code for the emojiTrie initialization is never called when the test is run automatically.

What does the occurrence of the issue mean for the software quality of the project?

This impacts directly on Reliability. The uncovered code is a "blind spot" of this project. There is no automated test that touch this line, so any kind of bugs in this line (like issue with `CodePointTrie.create()`) would not be discovered until production. It means that the testing approach for the commons package is not adequate.

Excessive Open Issues (Technical Debt Density)

There are 2,505 Open Issues reported here in Analysis report for this one class. This is normally a result of following "Don't Repeat Yourself" (DRY) principle or having thousands of minor violations as in formatting or naming in a file generated or containing a many lot of data, A significant Maintainability issue. If there are thousands of issues in one file, there will have "alert fatigue. The amount of errors will be very overwhelming and developers are likely to turn a blind eye to this static analysis tool altogether.

What does the occurrence of the issue mean for the software quality of the project?

This makes it kind of difficult to see important bugs in the minor code that creep in, and the technical debt builds up and is more expensive to pay back over time.

Discuss what the analysis results indicate about the software quality of the project.

The overall analysis result of our SIRIUS kernel shows there is lot of Technical Debt in the project. The project appears to be kind of well structured, although there are 2,505 issues within only one class and no uncovered code in the common utilities, indicating that project might not be uniformly well-controlled. Kernel also depends on a number of very complex "God Objects" (classes) and has no granular exception handling. The analysis results show that somewhat software might be functional, but not probably not very maintainable in the long-term. The team working on this project should have automated coverage requirements for core utilities and focus on refactoring classes having a lot of data like Emojis.java. This is to less-en the noise in final quality reports. This could also enable developers to concentrate on more significant defects in reliability and security of their products.

Practical Project Tasks 02: Fuzzing

Task 2.1 Setup Your Project for Fuzzing

```
Semantic Fuzzing with Zest

Test name:          sirius.kernel.commons.AmountFuzzer#testAmountScientificString
Instrumentation:     Janala
Results directory:   /Users/anisha/Downloads/ThirdSem/Testing/sirius-kernel/target/fuzz-results/sirius.kernel.commons.AmountFuzzer/testAmountScientificString
Elapsed time:        6m 10s (no time limit)
Number of executions: 1.687.728 (no trial limit)
Valid inputs:        1.687.728 (100,00%)
Cycles completed:    1687
Unique failures:     0
Queue size:          1 (1 favored last cycle)
Current parent input: 0 (favored) {727/1000 mutations}
Execution speed:      3.996/sec now | 4.558/sec overall
Total coverage:       8 branches (0,01% of map)
Valid coverage:       8 branches (0,01% of map)
^C
anisha@Mac sirius-kernel %
```


2. Interpretation of Test Results

Observation

- a. testSafeConversions had a lot of java.lang.NumberFormatException crashes after the initial fuzzing runs. The stack traces indicated that these exceptions are being thrown from within the Value class as a result of trying to use standard Java parsing methods on fuzzed input.
- b. testBooleanParsing method returned true all times that it was called (0 failures).

Interpretation

- a. Numeric types couldnt be parsed internally by the class. So, the library threw exception that has to be handled by developer. This force extra error-handling code, although the purpose of the class is to be a safe convenience helper, so that's a problem. Value class should be updated to include try-catch blocks for numeric parsing, so that errors do not throw system exceptions, but rather return default value that was passed in.

I added a try-catch block to test for these exceptions. This will allow the fuzzer to continue testing other parts, but also provide an example of how this library has a weakness with regard to its numeric conversion.

- b. The logic behind the boolean interpretation is robust. The library is able to normalize the strings by trimming and making them lowercase, which is good for consistent string processing for common inputs like "true" or "yes" or "1" or "on". This means that the boolean parsing module is really well-implemented, and is resistant to common variations.

b. sirius.kernel.commons.MultiMapFuzzer

```
Semantic Fuzzing with Zest
-----
Test name:          sirius.kernel.commons.MultiMapFuzzer#testPutAndGet
Instrumentation:     Janala
Results directory:   /Users/anisha/Downloads/ThirdSem/Testing/sirius-kernel/target/fuzz-results/sirius.kernel.commons.MultiMapFuzzer/testPutAndGet
Elapsed time:        8s (no time limit)
Number of executions: 36.897 (no trial limit)
Valid inputs:        36.897 (100,00%)
Cycles completed:    36
Unique failures:     0
Queue size:          1 (1 favored last cycle)
Current parent input: 0 (favored) {896/1000 mutations}
Execution speed:     4.696/sec now | 4.241/sec overall
Total coverage:       3 branches (0,00% of map)
Valid coverage:       3 branches (0,00% of map)
^C
anisha@Mac sirius-kernel %
```

```
Semantic Fuzzing with Zest
-----
Test name:          sirius.kernel.commons.MultiMapFuzzer#testStructuralConsistency
Instrumentation:     Janala
Results directory:   /Users/anisha/Downloads/ThirdSem/Testing/sirius-kernel/target/fuzz-results/sirius.kernel.commons.MultiMapFuzzer/testStructuralConsistency
Elapsed time:        17s (no time limit)
Number of executions: 72.222 (no trial limit)
Valid inputs:        72.222 (100,00%)
Cycles completed:    44
Unique failures:     0
Queue size:          4 (3 favored last cycle)
Current parent input: 0 (favored) {413/500 mutations}
Execution speed:     4.380/sec now | 4.149/sec overall
Total coverage:       8 branches (0,01% of map)
Valid coverage:       8 branches (0,01% of map)
^C
anisha@Mac sirius-kernel %
```

1. Test Methodology

The MultiMap is a specialized collection with multiple values being mapped to the same key.

Unlike StandardMap, it needs internal logic in order to handle the aggregation of values (which are List or Set).

- a. testPutAndGet: Test to verify data integrity. I chose this because the main requirement of MultiMap is that all values inserted must be retrievable. So, arbitrary strings are used for fuzzing to be sure that the collection handling don't lose data or report wrong elements in the collection even when the number of elements added is in high frequency.
- b. testStructuralConsistency: This test is for checking structural invariants. If several key-value pairs are added, the internal keySet needs to be synchronized to number of distinct keys added. I used a random list of keys and cycling throughout the values, causing the structure to handle a possible collision and different number of entries.

2. Interpretation of Test Results

Observations

- a. After few executions, both testPutAndGet and testStructuralConsistency have 0 unique failures. The branch coverage was very small (3 branches), meaning it's probably a clean wrapper around the standard collection types.

Interpretation

- a. The level of stability of these tests suggests that MultiMap is indeed following its functional contract. It can reliably maintain the life cycle of keys and values, including the ability to map multiple items to single key.
- b. MultiMap could be a production ready collection class, since the fuzzer didn't crash and didn't cause any assertion to fail when trying to test edge cases such as empty lists or trying to re-insert a key too many times. It could be safe to assume that internal list synchronization logic is correct and won't cause any corruption error in tested scenarios.

c. sirius.kernel.commonns.TrieFuzzer

```
Semantic Fuzzing with Zest

Test name:      sirius.kernel.commonns.TrieFuzzer#testTriePutAndGet
Instrumentation: Janala
Results directory: /Users/anisha/Downloads/ThirdSem/Testing/sirius-kernel/target/fuzz-results/sirius.kernel.commonns.TrieFuzzer/testTriePutAndGet
Elapsed time:   9s (no time limit)
Number of executions: 39.855 (no trial limit)
Valid inputs:   39.855 (100,00%)
Cycles completed: 39
Unique failures: 0
Queue size:     1 (1 favored last cycle)
Current parent input: 0 (favored) {854/1000 mutations}
Execution speed: 4.616/sec now | 4.151/sec overall
Total coverage:  2 branches (0,00% of map)
Valid coverage:  2 branches (0,00% of map)

^C
```



```
Semantic Fuzzing with Zest

Test name:      sirius.kernel.commonns.TrieFuzzer#testPrefixMatching
Instrumentation: Janala
Results directory: /Users/anisha/Downloads/ThirdSem/Testing/sirius-kernel/target/fuzz-results/sirius.kernel.commonns.TrieFuzzer/testPrefixMatching
Elapsed time:   2s (no time limit)
Number of executions: 6.937 (no trial limit)
Valid inputs:   6.932 (99,93%)
Cycles completed: 6
Unique failures: 1
Queue size:     1 (1 favored last cycle)
Current parent input: 0 (favored) {936/1000 mutations}
Execution speed: 4.256/sec now | 3.193/sec overall
Total coverage:  2 branches (0,00% of map)
Valid coverage:  2 branches (0,00% of map)

^C
anisha@Mac sirius-kernel %
```

1. Test Methodology

Trie(Prefix Tree) is a complex data structure used to efficiently search for strings. It's because of the requirements of overlapping strings. eg: "cat", "cats", "cater".

- testTriePutAndGet: This test implements the basic functional contract. It guarantees that, if a random string is added to the Trie, then calling get() will return the value of that string.
- testPrefixMatching: It's for structural integrity. I wanted to test that the Trie would correctly identify if the key was in the Trie or not, including cases like overlapping prefixes with the keys I added and the key I searched for.

2. Interpretation of Test Results

Observations

- testTriePutAndGet completed with 0 failures.
- testPrefixMatching flagged 1 unique Failure.

Interpretation

- The single failure in testPrefixMatching is very important. For a Trie, this typically happens when its prefix and key-to-value associations are wrong (e.g. requesting "cat" and getting back the value for "cats"). It's probably because of the edge case with partial string matching.

Component	Stability	Finding
Value	Low	Found robustness issues (Unhandled `NumberFormatException`s)
MultiMap	High	No issues found; structure is stable
Trie	Medium	Found an edge case failure in prefix matching logic

The validations of the core components of sirius-kernel library were achieved with this fuzzing. I found that the data structures (MultiMap) are fairly robust, but some utility classes (Value) and advanced search data structures (Trie) have certain size or corner cases where they are vulnerable. The maintainers of the sirius-kernel are recommended to take a look at the `NumberFormatException` propagation in `Value.java` and the prefix matching in `Trie.java` to make the library more robust.